

Agente Autonomo

v13 → v14

Optimizacion de Recursos, Arquitectura Modular
y Mejoras Criticas

RESUMEN

Analisis completo de los problemas criticos del agente v13 (2397 lineas monoliticas) y plan detallado para migrar a una arquitectura modular con 10+ modulos, cache LRU, analisis de proyectos profundo, persistencia binaria de vectores, y eliminacion de excepciones silenciosas.

Plan de Refactoring: Agente Autonomo v13 a v14

Optimizacion de Recursos, Arquitectura Modular y Mejoras Criticas

1. Diagnostico del Estado Actual (v13)	3
1.1 Problemas Criticos Identificados	3
1.2 Detalle del Problema: analizar_proyecto() Superficial	4
1.3 Detalle: _llm_generate() - Tres Estrategias Redundantes	4
1.4 Detalle: Excepciones Silenciosas	4
2. Arquitectura Modular Propuesta (v14)	5
2.1 Estructura de Directorios	5
2.2 Mapa de Responsabilidades	5
2.3 Patron de Registro de Herramientas (Decorator)	6
3. analizar_proyecto() Mejorado - Analisis Profundo	8
3.1Codigo Completo del Nuevo analizar_proyecto()	8
4. Optimizaciones de Recursos	12
4.1 Cache de Embeddings: FIFO a LRU	12
4.2 Similitud Coseno con Numpy Opcional	13
4.3 Simplificacion de _llm_generate()	14
4.4 Optimizacion de buscar_exe() con Cache Persistente	15
5. Mejoras de Calidad deCodigo	17
5.1 Eliminar Excepciones Silenciosas	17
5.2 Reducir Estado Global	17
5.3 Mejora de Persistencia de Vectores	18
6. Plan de Migracion: v13 a v14	19
6.1 Fases de Migracion	19
6.2 Estrategia de Compatibilidad	20

6.3 Alineacion con el Roadmap de 5 Fases	20
7. Codigo Refactorizado: config.py	21
8. Resumen de Mejoras y Proximos Pasos	23
8.1 Resumen de Mejoras por Categoria	23
8.2 Proximos Pasos Recomendados	23

1. Diagnostico del Estado Actual (v13)

El archivo `app_auto_pro.py` contiene actualmente 2,397 líneas de código en un único archivo monolítico. Si bien la versión 13 introdujo mejoras significativas como la Triple Memoria, herramientas de URL, y detección automática de modelos, la arquitectura actual presenta problemas estructurales que limitan la escalabilidad, el mantenimiento y la eficiencia del agente. Este diagnóstico identifica los problemas críticos que deben resolverse en la migración a v14, priorizando aquellos que afectan directamente el rendimiento y la calidad de las respuestas del agente.

1.1 Problemas Críticos Identificados

Problema	Severidad	Líneas	Impacto
Archivo monolítico (2,397 líneas)	CRÍTICO	1-2397	Imposible testear, mantener o escalar
<code>analizar_proyecto()</code> superficial	CRÍTICO	354-388	Análisis de repos sin profundidad
<code>_llm_generate()</code> excesivamente complejo	ALTO	1658-1801	140+ líneas con 3 estrategias redundantes
Excepciones silenciosas (<code>bare except:</code>)	ALTO	Múltiple	Errores ocultos imposibles de debuggear
Estado global sin control	ALTO	Múltiple	6+ variables globales mutables
Cache de embeddings FIFO (no LRU)	MEDIO	1090-1095	Eviction ineficiente de cache
similitud coseno en Python puro	MEDIO	1102-1111	Lento con vectores grandes (768 dim)
<code>buscar_exe()</code> escaneo de disco lento	MEDIO	500-564	Escanea Program Files completo
Persistencia JSON para vectores	MEDIO	1150-1170	Lento con muchos vectores
Duplicación: status Ollama en sidebar	BAJO	2272-2300	Dos bloques casi idénticos

Tabla 1. Inventario de problemas identificados en v13

1.2 Detalle del Problema: analizar_proyecto() Superficial

Este es el problema mas visible para el usuario. Cuando el agente analiza un repositorio como nexu-io/open-design, la funcion actual solo verifica si existen ciertos archivos (package.json, tsconfig.json, etc.) y lista la estructura de directorios. No lee el contenido de los archivos, no analiza dependencias, no identifica patrones arquitectonicos y no extrae informacion significativa. El resultado es un analisis que dice "Node.js + Git + README" cuando deberia identificar un monorepo pnpm con 259+ skills, soporte MCP, y arquitectura de plugins.

Codigo actual (v13):

```
def analizar_proyecto(ruta: str) -> str:
    # ... solo hace os.walk() y checks de existencia ...
    checks = {
        "package.json": "Node.js", "tsconfig.json": "TypeScript",
        "next.config.js": "Next.js", ".git": "Git", "README.md": "README",
    }
    for fname, desc in checks.items():
        if os.path.exists(os.path.join(ruta, fname)):
            resultado += f" - {desc} ({fname})\n"
    return resultado # Superficial: no lee contenidos!
```

1.3 Detalle: _llm_generate() - Tres Estrategias Redundantes

La funcion _llm_generate() intenta tres estrategias secuenciales para conectar con Ollama: (1) usar conexion cacheada, (2) buscar con ollama.Client, y (3) HTTP directo. Cada estrategia itera sobre modelos y hosts, resultando en hasta 12 combinaciones probadas. Aunque la primera llamada exitosa se cachea, la logica tiene caminos que nunca se ejecutan (por ejemplo, el cliente global de ollama nunca se usa) y el fallback a HTTP deberia ser innecesario si el paquete ollama esta instalado. La complejidad ciclomatica de esta funcion es excesiva y deberia simplificarse separando la logica de conexion de la logica de generacion.

1.4 Detalle: Excepciones Silenciosas

El patron except: pass aparece mas de 20 veces en el codigo. Esto es extremadamente peligroso porque oculta errores que deberian ser diagnosticados. Por ejemplo, si _get_embedding() falla silenciosamente, el VectorStore funciona sin embeddings (usando fallback a texto), pero el usuario nunca sabe que la busqueda semantica esta deshabilitada. Similarmente, si save_session() falla, se pierde la sesion sin notificacion. Cada bloque except debe capturar la excepcion especifica y

registrarla en el log para diagnostico.

2. Arquitectura Modular Propuesta (v14)

La refactorizacion principal consiste en dividir el archivo monolitico en modulos con responsabilidades claras. Cada modulo encapsula un dominio funcional especifico, facilitando testing unitario, mantenimiento y extension futura. La estructura propuesta sigue el patron de separacion de responsabilidades donde la configuracion, las herramientas, la memoria, el agente y la interfaz son independientes entre si, comunicandose mediante interfaces bien definidas.

2.1 Estructura de Directorios

```
agente_v14/
__init__.py
config.py          # Constantes, rutas, modelos preferidos
security.py        # Validacion de paths, sanitizacion, comandos peligrosos
ollama_client.py   # Conexion LLM, embeddings, deteccion de modelos
tools/
  __init__.py
  base.py          # Registry de herramientas, TOOL_SCHEMAS auto-generation
  system.py        # ejecutar_comando, procesos_activos, matar_proceso
  files.py         # leer_archivo, escribir_archivo, listar_archivos, buscar_en_archivos
  web.py           # abrir_url, buscar_youtube, buscar_web
  apps.py          # abrir_aplicacion, buscar_exe, buscar_en_start_menu
  dev.py           # generar_codigo, analizar_proyecto, clonar_repositorio
  schemas.py       # TOOL_SCHEMAS (auto-generado desde decorators)
memory/
  __init__.py
  vector_store.py  # VectorStore con numpy opcional
  triple_memory.py # TripleMemory con contexto enriquecido
  learning.py      # LearningSystem (correcciones, conocimiento)
agent/
  __init__.py
  react.py         # ReactAgent (motor principal)
  prompts.py      # SYSTEM_PROMPT, JSON_TOOLS_PROMPT
ui/
  __init__.py
  streamlit_app.py # Interfaz Streamlit
main.py           # Punto de entrada
```

2.2 Mapa de Responsabilidades

Modulo	Responsabilidades	Lineas estimadas
config.py	Constantes, rutas, modelos, sitios web, variables de entorno	~120
security.py	Validacion de paths, sanitizacion, comandos peligrosos/seguros	~80
ollama_client.py	Conexion LLM, embeddings, deteccion de modelos, cache	~200
tools/system.py	Ejecucion de comandos, procesos del sistema	~80
tools/files.py	Operaciones de archivos: leer, escribir, listar, buscar	~120
tools/web.py	URLs, YouTube, busqueda web	~80
tools/apps.py	Aplicaciones de escritorio, Start Menu, EXEs	~140
tools/dev.py	Generacion de codigo, analisis de proyectos, clonar, instalar	~200
memory/vector_store.py	VectorStore con busqueda semantica	~150
memory/triple_memory.py	Triple Memoria con contexto enriquecido	~200
memory/learning.py	Sistema de aprendizaje y correcciones	~80
agent/react.py	Motor ReAct principal	~200
agent/prompts.py	System prompts y templates	~60
ui/streamlit_app.py	Interfaz grafica Streamlit	~200
main.py	Punto de entrada, orquestacion	~30

Tabla 2. Distribucion de responsabilidades en la arquitectura modular

2.3 Patron de Registro de Herramientas (Decorator)

En v13, las herramientas se definen en tres lugares distintos: la funcion Python, el TOOL_SCHEMAS (JSON manual), y el TOOL_FUNCTIONS (diccionario manual). Esto significa que agregar una herramienta nueva requiere editar tres archivos diferentes, lo cual es propenso a errores. El patron propuesto usa un decorator que registra automaticamente la funcion, genera el schema JSON y la agrega al registro,

eliminando la duplicacion y garantizando consistencia entre la definicion y el schema.

```
# tools/base.py - Registry automatico con decorator
import inspect
from functools import wraps

TOOL_REGISTRY = {} # name -> {func, schema, description}

def tool(name: str, description: str, params: dict, required: list = None):
    """Decorator que registra una herramienta automaticamente."""
    def decorator(func):
        schema = {
            "type": "function",
            "function": {
                "name": name,
                "description": description,
                "parameters": {
                    "type": "object",
                    "properties": params,
                    "required": required or []
                }
            }
        }
        TOOL_REGISTRY[name] = {
            "func": func,
            "schema": schema,
            "description": description
        }
        @wraps(func)
        def wrapper(*args, **kwargs):
            return func(*args, **kwargs)
        return wrapper
    return decorator

# Uso en tools/system.py:
@tool(
    name="ejecutar_comando",
    description="Ejecuta un comando en la terminal. Peligrosos requieren confirmacion.",
    params={"comando": {"type": "string", "description": "Comando a ejecutar"},
            "confirmar_peligroso": {"type": "boolean", "description": "True si confirmado"}},
    required=["comando"]
)

def ejecutar_comando(comando: str, confirmar_peligroso: bool = False) -> str:
    # ... implementacion ...
    pass

# Obtener schemas y funciones automaticamente:
def get_tool_schemas(): return [r["schema"] for r in TOOL_REGISTRY.values()]
def get_tool_functions(): return {n: r["func"] for n, r in TOOL_REGISTRY.items()}
```


3. analizar_proyecto() Mejorado - Analisis Profundo

La mejora mas critica para la experiencia del usuario. El nuevo `analizar_proyecto()` lee realmente los archivos del proyecto, analiza dependencias, identifica frameworks, detecta patrones arquitectonicos y genera un reporte detallado. El analisis se realiza en tres fases: (1) exploracion del arbol de directorios, (2) lectura e interpretacion de archivos clave, y (3) sintesis del reporte final. Este enfoque garantiza que el agente pueda proporcionar analisis significativos como el que se esperaba con el repositorio open-design, identificando monorepos, arquitecturas de plugins, y stacks tecnologicos completos.

3.1 Codigo Completo del Nuevo `analizar_proyecto()`

```
def analizar_proyecto(ruta: str) -> str:
    """Analisis PROFUNDO de un proyecto. Lee archivos, detecta
    frameworks, dependencias, arquitectura y genera reporte completo."""

    # Resolver ruta
    if not os.path.exists(ruta):
        alt = os.path.join(REPOS_DIR, ruta) if not os.path.isabs(ruta) else ruta
        if os.path.exists(alt):
            ruta = alt
        else:
            return f"Directorio no existe: {ruta}"

    resultado = []
    resultado.append(f"ANALISIS PROFUNDO: {ruta}")
    resultado.append("=" * 50)

    # — FASE 1: Estructura de directorios —
    dir_info = _scan_directory(ruta, max_depth=4)
    resultado.append(f"\nESTRUCTURA ({dir_info['dirs']} dirs, {dir_info['files']} archivos):")
    resultado.append(dir_info['tree'][:2000])

    # — FASE 2: Lectura e interpretacion de archivos clave —
    tech_stack = []
    frameworks = []
    deps = {}
    arch_patterns = []
    key_files_content = {}

    # 2a. package.json → Node.js ecosystem
    pkg_path = _find_file(ruta, "package.json")
    if pkg_path:
        try:
            with open(pkg_path, "r", encoding="utf-8", errors="replace") as f:
                pkg = json.load(f)
```

```

tech_stack.append("Node.js")
deps["dependencies"] = list(pkg.get("dependencies", {}).keys())
deps["devDependencies"] = list(pkg.get("devDependencies", {}).keys())

# Detectar framework
all_deps = list(pkg.get("dependencies", {}).keys()) + \
            list(pkg.get("devDependencies", {}).keys())
fw_map = {
    "next": "Next.js", "react": "React", "vue": "Vue",
    "svelte": "Svelte", "express": "Express", "fastify": "Fastify",
    "astro": "Astro", "nuxt": "Nuxt", "angular": "Angular",
}
for dep, name in fw_map.items():
    if dep in all_deps:
        frameworks.append(name)

# Detectar monorepo
workspaces = pkg.get("workspaces", [])
if workspaces or os.path.exists(os.path.join(ruta, "pnpm-workspace.yaml")):
    arch_patterns.append("Monorepo")
    if os.path.exists(os.path.join(ruta, "pnpm-workspace.yaml")):
        arch_patterns.append("pnpm workspace")

# Detectar tipo de proyecto
if "tslib" in all_deps or "typescript" in all_deps:
    tech_stack.append("TypeScript")
if pkg.get("type") == "module":
    tech_stack.append("ESM")

# Scripts importantes
scripts = pkg.get("scripts", {})
if scripts:
    resultado.append(f"\nSCRIPTS: {' '.join(scripts.keys())}")

key_files_content["package.json"] = pkg
except Exception as e:
    resultado.append(f" [WARN] Error leyendo package.json: {e}")

# 2b. pnpm-workspace.yaml
pnpm_ws = _find_file(ruta, "pnpm-workspace.yaml")
if pnpm_ws:
    try:
        content = _safe_read(pnpm_ws, max_chars=2000)
        if content:
            arch_patterns.append("pnpm monorepo")
            resultado.append(f"\nWORKSPACE pnpm:\n{content[:500]}")
    except: pass

# 2c. requirements.txt / pyproject.toml → Python ecosystem
req_path = _find_file(ruta, "requirements.txt")
if req_path:
    try:
        content = _safe_read(req_path, max_chars=3000)
        tech_stack.append("Python")

```

```

        pip_deps = [l.strip().split("==")[0].split(">=")[0]
                     for l in content.split("\n") if l.strip() and not l.startswith("#")]
        deps["pip"] = pip_deps
        fw_py = {"django": "Django", "flask": "Flask", "fastapi": "FastAPI",
                 "streamlit": "Streamlit", "pydantic": "Pydantic"}
        for dep, name in fw_py.items():
            if any(dep in d.lower() for d in pip_deps):
                frameworks.append(name)
    except: pass

pyproject = _find_file(ruta, "pyproject.toml")
if pyproject:
    tech_stack.append("Python (pyproject)")
    content = _safe_read(pyproject, max_chars=2000)
    if "[tool.poetry]" in content:
        arch_patterns.append("Poetry")

# 2d. README.md → Descripción del proyecto
readme = _find_file(ruta, "README.md")
if readme:
    try:
        content = _safe_read(readme, max_chars=4000)
        lines = content.split("\n")
        # Buscar título
        title = next((l.strip("# ").strip() for l in lines
                      if l.startswith("# ")), "")
        if title:
            resultado.append(f"\nPROYECTO: {title}")
        # Buscar badges/descripción
        desc_lines = [l.strip() for l in lines[1:10]
                      if l.strip() and not l.startswith("#")
                      and not l.startswith("![" ) and not l.startswith("![")
                      and len(l.strip()) > 20]
        if desc_lines:
            resultado.append(f"DESCRIPCION: {desc_lines[0][:300]}")
    except: pass

# 2e. Docker / CI/CD
if _find_file(ruta, "Dockerfile"):
    tech_stack.append("Docker")
if _find_file(ruta, "docker-compose.yml"):
    arch_patterns.append("Docker Compose")
if _find_file(ruta, ".github/workflows"):
    arch_patterns.append("GitHub Actions CI/CD")
if _find_file(ruta, ".gitlab-ci.yml"):
    arch_patterns.append("GitLab CI/CD")

# 2f. MCP / Skills / Plugins (patrones avanzados)
if os.path.exists(os.path.join(ruta, "skills")):
    n_skills = len(os.listdir(os.path.join(ruta, "skills")))
    arch_patterns.append(f"Skills system ({n_skills}+ skills)")
if os.path.exists(os.path.join(ruta, "plugins")):
    n_plugins = len(os.listdir(os.path.join(ruta, "plugins")))
    arch_patterns.append(f"Plugins ({n_plugins}+ plugins)")

```

```

if os.path.exists(os.path.join(ruta, "packages")):
    n_pkgs = len([d for d in os.listdir(os.path.join(ruta, "packages"))
                  if os.path.isdir(os.path.join(ruta, "packages", d))])
    arch_patterns.append(f"Multi-paquetes ({n_pkgs} packages)")

# — FASE 3: Sintesis del reporte —
if tech_stack:
    resultado.append(f"\nTECNOLOGIAS: {' ', '.join(set(tech_stack))}")
if frameworks:
    resultado.append(f"FRAMEWORKS: {' ', '.join(set(frameworks))}")
if arch_patterns:
    resultado.append(f"ARQUITECTURA: {' ', '.join(arch_patterns)}")

# Dependencias principales
for dep_type, dep_list in deps.items():
    if dep_list:
        shown = dep_list[:15]
        extra = f" +{len(dep_list)-15} mas" if len(dep_list) > 15 else ""
        resultado.append(f"\n{dep_type.upper()}: {' ', '.join(shown)}{extra}")

# Resumen de directorios principales
top_dirs = [d for d in os.listdir(ruta)
             if os.path.isdir(os.path.join(ruta, d)) and not d.startswith(".")]
if top_dirs:
    resultado.append(f"\nDIRECTORIOS PRINCIPALES: {' ', '.join(sorted(top_dirs)[:15])}")

return "\n".join(resultado)

# — Funciones auxiliares —
def _find_file(root: str, filename: str) -> str:
    """Busca un archivo en el directorio raiz (no recursivo)."""
    path = os.path.join(root, filename)
    return path if os.path.exists(path) else ""

def _safe_read(path: str, max_chars: int = 4000) -> str:
    """Lee un archivo de forma segura con truncado."""
    try:
        with open(path, "r", encoding="utf-8", errors="replace") as f:
            content = f.read(max_chars)
        return content
    except Exception:
        return ""

def _scan_directory(ruta: str, max_depth: int = 4) -> dict:
    """Escanea estructura de directorios con profundidad limitada."""
    dirs_count = 0
    files_count = 0
    tree_lines = []

    for root, dirs, files in os.walk(ruta):
        level = root.replace(ruta, "").count(os.sep)
        if level >= max_depth:
            dirs.clear()

```

```

        continue
    indent = " " * level
    dirname = os.path.basename(root) or root
    tree_lines.append(f"{indent}{dirname}/")
    dirs_count += 1
    subindent = " " * (level + 1)
    for f in sorted(files)[:20]:
        tree_lines.append(f"{subindent}{f}")
        files_count += 1
    if len(files) > 20:
        tree_lines.append(f"{subindent}... +{len(files)-20} mas")

    return {
        "tree": "\n".join(tree_lines[:100]),
        "dirs": dirs_count,
        "files": files_count
    }

```

4. Optimizaciones de Recursos

Dado que el agente corre en una maquina con RTX 3060 (12GB VRAM) y 16GB RAM, la optimizacion de recursos es critica. El objetivo es maximizar la eficiencia de cada componente: reduccion del consumo de RAM, aceleracion de la busqueda semantica, optimizacion del uso de GPU, y reduccion de llamadas redundantes a Ollama. Las siguientes optimizaciones se priorizan por impacto en el rendimiento percibido por el usuario.

4.1 Cache de Embeddings: FIFO a LRU

El cache actual de embeddings usa un patron FIFO (First In, First Out) que elimina las entradas mas viejas cuando se llena. Esto es ineficiente porque las entradas mas viejas pueden ser las mas utilizadas (por ejemplo, embeddings de conocimiento base que se consultan frecuentemente). El patron LRU (Least Recently Used) elimina las entradas que no se han usado recientemente, lo cual es mucho mas eficiente. Python ofrece `OrderedDict` que ya soporta LRU de forma nativa moviendo las claves al final en cada acceso.

```

# ANTES (v13): FIFO - elimina las mas viejas, no las menos usadas
if len(_EMBED_CACHE) >= _EMBED_CACHE_MAX:
    oldest_keys = list(_EMBED_CACHE.keys())[:_EMBED_CACHE_MAX // 2]
    for k in oldest_keys:
        del _EMBED_CACHE[k]

# DESPUES (v14): LRU con OrderedDict - mueve al final al acceder
from collections import OrderedDict

```

```

class LRUEmbedCache:
    def __init__(self, max_size=200):
        self._cache = OrderedDict()
        self._max_size = max_size

    def get(self, key):
        if key in self._cache:
            self._cache.move_to_end(key) # Marcar como recién usado
            return self._cache[key]
        return None

    def put(self, key, value):
        if key in self._cache:
            self._cache.move_to_end(key)
        else:
            self._cache[key] = value
            if len(self._cache) > self._max_size:
                self._cache.popitem(last=False) # Eliminar menos reciente

```

4.2 Similitud Coseno con Numpy Opcional

La implementación actual de similitud coseno usa Python puro, iterando sobre todos los elementos del vector. Con vectores de 768 dimensiones (nomic-embed-text), cada comparación requiere 768 multiplicaciones y sumas. Cuando el VectorStore tiene 100+ entradas, una búsqueda requiere 100 x 768 operaciones. Numpy puede acelerar esto por un factor de 10-50x usando operaciones vectorizadas C/Fortran. La solución propuesta detecta si numpy está disponible y lo usa, con fallback a Python puro si no lo está.

```

# Detección de numpy con fallback graceful
try:
    import numpy as np
    HAS_NUMPY = True
except ImportError:
    HAS_NUMPY = False

def cosine_similarity(vec1, vec2):
    """Similitud coseno optimizada con numpy si disponible."""
    if not vec1 or not vec2 or len(vec1) != len(vec2):
        return 0.0
    if HAS_NUMPY:
        a, b = np.array(vec1), np.array(vec2)
        denom = np.linalg.norm(a) * np.linalg.norm(b)
        return float(np.dot(a, b) / denom) if denom > 0 else 0.0
    else:
        dot = sum(a * b for a, b in zip(vec1, vec2))
        norm1 = sum(a * a for a in vec1) ** 0.5
        norm2 = sum(b * b for b in vec2) ** 0.5
        return dot / (norm1 * norm2) if norm1 and norm2 else 0.0

```

```
def batch_cosine_similarity(query_vec, vectors_dict):
    """Busqueda semantica en batch - MUCHO mas rapida con numpy."""
    if not HAS_NUMPY or not vectors_dict:
        # Fallback: una por una
        return {k: cosine_similarity(query_vec, v) for k, v in vectors_dict.items()}

    ids = list(vectors_dict.keys())
    matrix = np.array([vectors_dict[k] for k in ids]) # (N, dim)
    query = np.array(query_vec) # (dim,)

    # Producto punto vectorizado: una sola operacion
    dots = matrix @ query
    norms = np.linalg.norm(matrix, axis=1) * np.linalg.norm(query)
    scores = np.where(norms > 0, dots / norms, 0.0)

    return {k: float(s) for k, s in zip(ids, scores)}
```

4.3 Simplificacion de _llm_generate()

La funcion actual intenta 3 estrategias con multiples bucles anidados. La version simplificada usa un patron de conexion persistente con retry simple: primero intenta con la conexion cacheada, si falla busca una nueva, y si todo falla usa HTTP directo como ultimo recurso. Eliminamos la duplicacion de bucles y reducimos la complejidad ciclomatica de 25+ a menos de 10.

```
# ollama_client.py - LLM Client simplificado
import json, logging, urllib.request
from typing import Optional, Union

logger = logging.getLogger("agente.llm")

class OllamaClient:
    """Cliente Ollama con conexion persistente y retry simple."""

    def __init__(self, host="http://localhost:11434"):
        self.host = host
        self._client = None
        self._model = None
        self._embed_model = None

    def _get_client(self):
        """Obtiene o crea cliente ollama."""
        if self._client is not None:
            return self._client
        try:
            import ollama
            self._client = ollama.Client(host=self.host)
            return self._client
        except ImportError:
            return None
```

```

def chat(self, messages, model=None, tools=None, timeout=120) -> str:
    """Chat simplificado: client -> HTTP directo."""
    model = model or self.detect_model()

    # Estrategia 1: ollama Client
    client = self._get_client()
    if client:
        try:
            kwargs = {"model": model, "messages": messages}
            if tools:
                kwargs["tools"] = tools
            resp = client.chat(**kwargs)
            if tools:
                return resp # Return full response with tool_calls
            content = resp.get("message", {}).get("content", "")
            if content:
                return content
        except Exception as e:
            logger.warning(f"Client failed: {e}, trying HTTP")

    # Estrategia 2: HTTP directo
    return self._http_chat(messages, model, timeout)

def _http_chat(self, messages, model, timeout):
    """Fallback HTTP directo."""
    data = json.dumps({
        "model": model, "messages": messages, "stream": False
    }).encode("utf-8")
    req = urllib.request.Request(
        f"{self.host}/api/chat", data=data,
        headers={"Content-Type": "application/json"}
    )
    with urllib.request.urlopen(req, timeout=timeout) as resp:
        result = json.loads(resp.read().decode("utf-8"))
        return result.get("message", {}).get("content", "")

def detect_model(self) -> str:
    """Detecta el mejor modelo disponible (cacheado)."""
    if self._model:
        return self._model
    # ... logica de deteccion (se ejecuta una sola vez) ...

def get_embedding(self, text: str) -> list:
    """Obtiene embedding con cache LRU."""
    # ... usa LRUEmbedCache internamente ...

# Singleton global
ollama_client = OllamaClient()

```

4.4 Optimizacion de buscar_exe() con Cache Persistente

La funcion `buscar_exe()` es la operacion mas lenta del agente porque escanea directorios enteros de Program Files. Con un cache TTL de 5 minutos en memoria, solo acelera busquedas repetidas dentro de la misma sesion. La mejora propone un cache persistente en disco que sobrevive entre sesiones, reduciendo el tiempo de busqueda de varios segundos a milisegundos para aplicaciones frecuentes como Chrome o VSCode.

```
# apps.py - Cache persistente para busqueda de ejecutables
import json, time, os

EXE_CACHE_FILE = os.path.join(LEARN_DIR, "exe_cache.json")

class ExeCache:
    """Cache persistente de rutas de ejecutables con TTL."""

    def __init__(self, cache_file=EXE_CACHE_FILE, ttl=3600):
        self._cache_file = cache_file
        self._ttl = ttl # 1 hora por defecto
        self._cache = self._load()

    def _load(self):
        try:
            if os.path.exists(self._cache_file):
                with open(self._cache_file, "r", encoding="utf-8") as f:
                    return json.load(f)
        except (json.JSONDecodeError, OSError):
            pass
        return {}

    def _save(self):
        try:
            with open(self._cache_file, "w", encoding="utf-8") as f:
                json.dump(self._cache, f, ensure_ascii=False, indent=2)
        except OSError:
            pass

    def get(self, name):
        entry = self._cache.get(name.lower())
        if not entry:
            return None
        # Verificar TTL y que el archivo aun existe
        if time.time() - entry["ts"] > self._ttl:
            del self._cache[name.lower()]
            return None
        if not os.path.exists(entry["path"]):
            del self._cache[name.lower()]
            return None
        return entry["path"]

    def put(self, name, path):
        self._cache[name.lower()] = {"path": path, "ts": time.time()}
        self._save()
```

```
exe_cache = ExeCache()
```

5. Mejoras de Calidad de Código

5.1 Eliminar Excepciones Silenciosas

Cada bloque `except: pass` debe ser reemplazado por un bloque que capture la excepción específica y la registre en el log. Esto es fundamental para poder diagnosticar problemas cuando el agente no funciona como se espera. El patrón es simple: en lugar de ignorar el error, registrarlo con nivel `WARNING` o `ERROR` según la severidad, y proporcionar suficiente contexto en el mensaje para entender que falló y por qué.

```
# ANTES (v13): Excepcion silenciosa - errores ocultos
def _load_index(self) -> list:
    try:
        if os.path.exists(self.index_file):
            with open(self.index_file, "r", encoding="utf-8") as f:
                return json.load(f)
    except: # Si JSON esta corrupto, nunca lo sabras!
        pass
    return []

# DESPUES (v14): Excepcion especifica con logging
def _load_index(self) -> list:
    try:
        if os.path.exists(self.index_file):
            with open(self.index_file, "r", encoding="utf-8") as f:
                return json.load(f)
    except json.JSONDecodeError as e:
        logger.warning(f"Index JSON corrupto, recreando: {e}")
    except OSError as e:
        logger.error(f"No se pudo leer index: {e}")
    return []
```

5.2 Reducir Estado Global

v13 tiene al menos 6 variables globales mutables: `_EMBED_CACHE`, `_ollama_client`, `_detected_model`, `_ollama_working_host`, `_ollama_working_method`, y `_ollama_embed_model`. Cada variable global es un potencial punto de falla en un programa concurrente y dificulta el testing. La solución es encapsular estas variables dentro de clases singleton que las gestionan internamente, exponiendo solo métodos de acceso seguro.

Variable Global	Problema	Solucion v14
_EMBED_CACHE	Diccionario mutable global	Encapsular en LRUEmbedCache
_ollama_client	Conexion sin control de ciclo de vida	Encapsular en OllamaClient singleton
_detected_model	Se resetea entre llamadas?	Cache permanente en OllamaClient
_exe_cache / _exe_cache_time	Dos dict separados sin sincronizacion	Encapsular en ExeCache
AGENT_MODEL / FALLBACK_MODEL	3 variables para 1 concepto	Unificar en OllamaClient
_ollama_working_method	Estado de conexion transitorio	OllamaClient maneja internamente

Tabla 3. Mapeo de variables globales a clases encapsuladas

5.3 Mejora de Persistencia de Vectores

El VectorStore actual guarda todos los vectores en un unico archivo JSON. Cuando el archivo crece (100+ entradas con vectores de 768 dimensiones cada una), la lectura y escritura se vuelve lenta porque debe cargar/descargar todo el archivo en cada operacion. La mejora propone un formato binario mas eficiente (usando struct o pickle) para los vectores, manteniendo el indice en JSON para busquedas rapidas, y usando carga lazy con flush diferido para reducir I/O.

```
# vector_store.py - Persistencia optimizada
import struct, pickle, os

class VectorStore:
    def __init__(self, store_dir):
        self.store_dir = store_dir
        self.index_file = os.path.join(store_dir, "index.json")
        self.vectors_file = os.path.join(store_dir, "vectors.bin") # Binario!
        self._vectors_cache = None
        self._dirty = False
        self._flush_interval = 5 # Flush cada 5 operaciones
        self._op_count = 0

    def _save_vectors(self, vectors):
        """Guarda vectores en formato binario (3-5x mas rapido que JSON)."""
        try:
            with open(self.vectors_file, "wb") as f:
                pickle.dump(vectors, f, protocol=pickle.HIGHEST_PROTOCOL)
            self._vectors_cache = vectors
        except OSError as e:
```

```

        logger.error(f"Error guardando vectores: {e}")

def _get_vectors(self):
    """Carga vectores con cache en memoria."""
    if self._vectors_cache is not None:
        return self._vectors_cache
    try:
        if os.path.exists(self.vectors_file):
            with open(self.vectors_file, "rb") as f:
                self._vectors_cache = pickle.load(f)
            return self._vectors_cache
    except (pickle.UnpicklingError, OSError) as e:
        logger.warning(f"Error cargando vectores binarios: {e}")
    self._vectors_cache = {}
    return self._vectors_cache

def _maybe_flush(self):
    """Flush diferido - no escribir en cada operacion."""
    self._op_count += 1
    if self._op_count >= self._flush_interval:
        self._flush()
        self._op_count = 0

```

6. Plan de Migracion: v13 a v14

La migracion de v13 a v14 debe realizarse de forma incremental para evitar romper funcionalidad existente. El plan sigue un enfoque de "strangler pattern" donde cada modulo se extrae del archivo monolitico y se reemplaza por un import, verificando que todo funcione despues de cada paso. Se recomienda mantener una copia del archivo v13 original como backup antes de comenzar.

6.1 Fases de Migracion

Fase	Tarea	Archivos Nuevos	Riesgo	Tiempo Est.
1	Extraer config.py y security.py	config.py, security.py	BAJO	30 min
2	Extraer ollama_client.py con LRU cache	ollama_client.py	MEDIO	1 hora
3	Mejorar analizar_proyecto() y extraer tools/	tools/dev.py + tools/	MEDIO	2 horas

4	Extraer memory/ con persistencia binaria	memory/	ALTO	2 horas
5	Extraer agent/ y prompts	agent/	MEDIO	1 hora
6	Extraer ui/ y limpiar main.py	ui/, main.py	BAJO	1 hora
7	Testing integracion completo	tests/	MEDIO	2 horas

Tabla 4. Fases de migracion con estimaciones de tiempo

6.2 Estrategia de Compatibilidad

Durante la migracion, el archivo `app_auto_pro.py` se mantiene funcional como punto de entrada, pero los modulos extraidos se importan desde los nuevos archivos. Esto permite retroceder cualquier cambio si algo falla. El patron de migracion es el siguiente: primero se crea el nuevo modulo, luego se agrega el import en el archivo principal, y finalmente se elimina el codigo duplicado del archivo original. Despues de cada fase, se ejecuta el agente y se verifica que las funciones basicas (chat, ejecutar comandos, abrir apps) sigan funcionando.

```
# Ejemplo: Fase 1 - Migracion de config.py
# Paso 1: Crear config.py con las constantes
# Paso 2: Reemplazar en app_auto_pro.py:
#   ANTES: PREFERRED_MODELS = ["qwen3:4b", ...]
#   DESPUES: from config import PREFERRED_MODELS, REPOS_DIR, ...
# Paso 3: Verificar que el agente arranca correctamente
# Paso 4: Eliminar las constantes duplicadas del archivo original
```

6.3 Alineacion con el Roadmap de 5 Fases

Esta refactorizacion es complementaria al roadmap de 5 fases del proyecto. No reemplaza ninguna fase, sino que prepara la arquitectura para implementarlas mas facilmente. La relacion entre la refactorizacion y cada fase del roadmap es la siguiente, demostrando como cada mejora estructural habilita o facilita las funcionalidades planificadas para el futuro del agente autonomo.

Fase Roadmap	Funcionalidad	Modulo v14 que lo Habilita
Fase 1: ReAct + FC	Tool calling nativo	tools/base.py (registry automatico)
Fase 2: Triple Memoria	Qdrant / Contexto	memory/ (ya preparado para swap)

Fase 3: MCP + Meta	Protocolo MCP	tools/ (extensible con plugins MCP)
Fase 4: Multimodal	Vision + Guardrails	ollama_client.py (soporte multimodal)
Fase 5: Multi-Agente	Workflow Engine	agent/react.py (composable)

Tabla 5. Alineacion del refactoring con el roadmap de 5 fases

7. Codigo Refactorizado: config.py

El modulo de configuracion centraliza todas las constantes, rutas y variables de configuracion que estaban dispersas en el archivo monolitico. Esto facilita cambiar configuraciones sin tocar la logica del agente y permite tener diferentes perfiles de configuracion (desarrollo, produccion, testing). El modulo tambien incluye la inicializacion de directorios y la carga de variables de entorno, consolidando la logica que antes estaba en el nivel superior del archivo.

```
# config.py - Configuracion centralizada del Agente v14
import os, platform
from pathlib import Path

# — Modelos preferidos en orden de prioridad —
PREFERRED_MODELS = [
    "qwen3:4b", "qwen3-coder", "qwen3-coder-next",
    "qwen3:30b-a3b", "qwen2.5:14b", "llama3.1:8b"
]

# — Parametros del agente —
MAX_REACT_ITERATIONS = 8
MAX_CONVERSATION_MEMORY = 20
CONTEXT_BUDGET_CHARS = 3000
AUTO_SAVE_INTERVAL = 5 # mensajes entre auto-saves
SESSION_TTL_HOURS = 24

# — Directorios —
if platform.system() == "Windows":
    REPOS_DIR = os.path.join(os.path.expanduser("~"), "Documents")
else:
    REPOS_DIR = os.path.join(os.path.expanduser("~"), "repos")

LEARN_DIR = os.path.join(os.path.expanduser("~"), ".ia-local", "learning")
os.makedirs(REPOS_DIR, exist_ok=True)
os.makedirs(LEARN_DIR, exist_ok=True)

# — Archivos de persistencia —
CORRECTIONS_FILE = os.path.join(LEARN_DIR, "corrections.json")
FEEDBACK_FILE = os.path.join(LEARN_DIR, "feedback.json")
```

```

PATTERNS_FILE = os.path.join(LEARN_DIR, "patterns.json")
KNOWLEDGE_FILE = os.path.join(LEARN_DIR, "knowledge.json")
SESSION_FILE = os.path.join(LEARN_DIR, "session.json")
EXE_CACHE_FILE = os.path.join(LEARN_DIR, "exe_cache.json")
LLM_ERRORS_LOG = os.path.join(LEARN_DIR, "llm_errors.log")

# — Seguridad —
COMANDOS_PELIGROSOS = [
    "rm -rf", "del /f /s /q", "format", "fdisk",
    "reg delete", "net user", "shutdown", "rmdir /s /q",
    "mkfs", "dd if=", "> /dev/sd", "curl | bash",
    "powershell -enc", "certutil", "bitsadmin",
]

COMANDOS_SEGUROS = [
    "git", "npm", "pip", "python", "node", "dir", "ls",
    "cat", "echo", "cd", "type", "find", "where", "which",
    "tasklist", "start", "open", "xdg-open",
    "pipenv", "poetry", "bun", "yarn", "cargo",
    "docker ps", "docker images", "docker compose",
]

# — Sitios web conocidos —
SITIOS_CONOCIDOS = {
    "youtube": "https://www.youtube.com",
    "google": "https://www.google.com",
    "gmail": "https://mail.google.com",
    "github": "https://github.com",
    "stackoverflow": "https://stackoverflow.com",
    "twitter": "https://twitter.com",
    "x": "https://x.com",
    "facebook": "https://www.facebook.com",
    "instagram": "https://www.instagram.com",
    "reddit": "https://www.reddit.com",
    "whatsapp web": "https://web.whatsapp.com",
    "netflix": "https://www.netflix.com",
    "spotify": "https://open.spotify.com",
    "chatgpt": "https://chat.openai.com",
    "notion": "https://www.notion.so",
    "figma": "https://www.figma.com",
}

# — Aliases de aplicaciones —
APP_ALIASES = {
    "chrome": "google chrome", "vscode": "visual studio code",
    "autocad": "autocad", "revit": "revit",
    "whatsapp": "whatsapp", "telegram": "telegram desktop",
    "word": "word", "excel": "excel", "powerpoint": "powerpoint",
    "notepad": "notepad",
}

# — Ollama —
OLLAMA_HOSTS = ["http://localhost:11434", "http://127.0.0.1:11434"]
EMBED_MODELS = ["nomic-embed-text", "mxbai-embed-large", "all-minilm"]

```

```
EMBED_CACHE_MAX = 200
EXE_CACHE_TTL = 3600 # 1 hora
```

8. Resumen de Mejoras y Proximos Pasos

La migracion de v13 a v14 representa un salto cualitativo en la arquitectura del agente autonomo. Las mejoras se agrupan en tres categorias principales: arquitectura (modularidad), rendimiento (optimizaciones de recursos), y calidad (eliminacion de excepciones silenciosas y estado global). Cada mejora ha sido disenada para ser compatible con el roadmap de 5 fases del proyecto, de modo que la refactorizacion no solo resuelve problemas actuales sino que prepara el terreno para las funcionalidades futuras como MCP, multimodal y multi-agente.

8.1 Resumen de Mejoras por Categoria

Categoria	Mejora	Impacto Esperado
Arquitectura	Archivo monolitico a 10+ modulos	Mantenibilidad 10x, testing posible
Arquitectura	Registry de herramientas con decorator	Agregar tools sin editar 3 archivos
Rendimiento	Cache LRU para embeddings	Cache hits +30% vs FIFO
Rendimiento	Similitud coseno con numpy	Busqueda 10-50x mas rapida
Rendimiento	Persistencia binaria de vectores	I/O 3-5x mas rapido
Rendimiento	Cache persistente para exe	Buscar apps en ms, no segundos
Rendimiento	_llm_generate simplificado	Menos latencia en primer contacto
Calidad	analizar_proyecto() profundo	Analisis de repos significativos
Calidad	Excepciones especificas con logging	Debuggable, no silencioso
Calidad	Estado global encapsulado en clases	Testeable, predecible
Calidad	Flush diferido en VectorStore	Menos I/O en operaciones frecuentes

Tabla 6. Resumen completo de mejoras planificadas

8.2 Proximos Pasos Recomendados

- **Paso 1:** Crear backup de app_auto_pro.py actual (v13) antes de cualquier cambio.
- **Paso 2:** Ejecutar Fase 1 de migracion (config.py + security.py) - riesgo bajo, se completa en 30 min.
- **Paso 3:** Reemplazar analizar_proyecto() con la version profunda - impacto inmediato visible.
- **Paso 4:** Extraer ollama_client.py con cache LRU - mejora rendimiento de embeddings.
- **Paso 5:** Extraer tools/ con registry de decorator - prepara para Fase 3 (MCP).
- **Paso 6:** Extraer memory/ con persistencia binaria - prepara para Fase 2 (Qdrant opcional).
- **Paso 7:** Completar extraccion de agent/ y ui/ - arquitectura modular completa.
- **Paso 8:** Agregar tests unitarios basicos para cada modulo - garantia de calidad.

El objetivo final es tener una base de codigo modular, testeable y eficiente que permita avanzar rapido en las fases 2-5 del roadmap. Cada modulo extraido es un componente independiente que puede mejorarse, reemplazarse o extenderse sin afectar los demas. La inversion de tiempo en esta refactorizacion (estimada en 8-10 horas) se recuperara con creces en las fases posteriores del proyecto, donde agregar funcionalidades sera cuestion de crear un nuevo archivo en el directorio correcto en vez de navegar 2400 lineas de codigo monolitico.